

Informe de Arquitectura y Dise o de Software     Sistema de Automatizaci  n IDIC

Proyecto: Automatizaci  n de Informes de Integridad Cient fica

Instituci  n: Instituto de Investigaci  n Cient fica (IDIC), Universidad de Lima

Rol: Arquitecto de Software Senior / Ingeniero Principal

Versi  n: 2.1 (Dise o Orientado a Servicios)

Fecha: 28 de abril de 2026

1. Resumen Ejecutivo

Este documento t  cnico detalla la arquitectura y metodolog  a de implementaci  n del **Sistema de Automatizaci  n de Informes de Integridad Cient fica**. El sistema ha sido dise ado bajo principios de **Ingenier  a de Software Senior**, priorizando el desacoplamiento, la mantenibilidad y la robustez en la integraci  n de datos heterog  neos.

La soluci  n automatiza la extracci  n de metadatos bibliom  tricos complejos desde m  ltiples fuentes (Scopus, WoS, JCR, Beall's List), transformando un flujo de trabajo manual propenso a errores en un proceso determinista y escalable que genera informes en formato Word (.docx) con precisi  n quir rgica.

2. Visi  n Arquitect nica

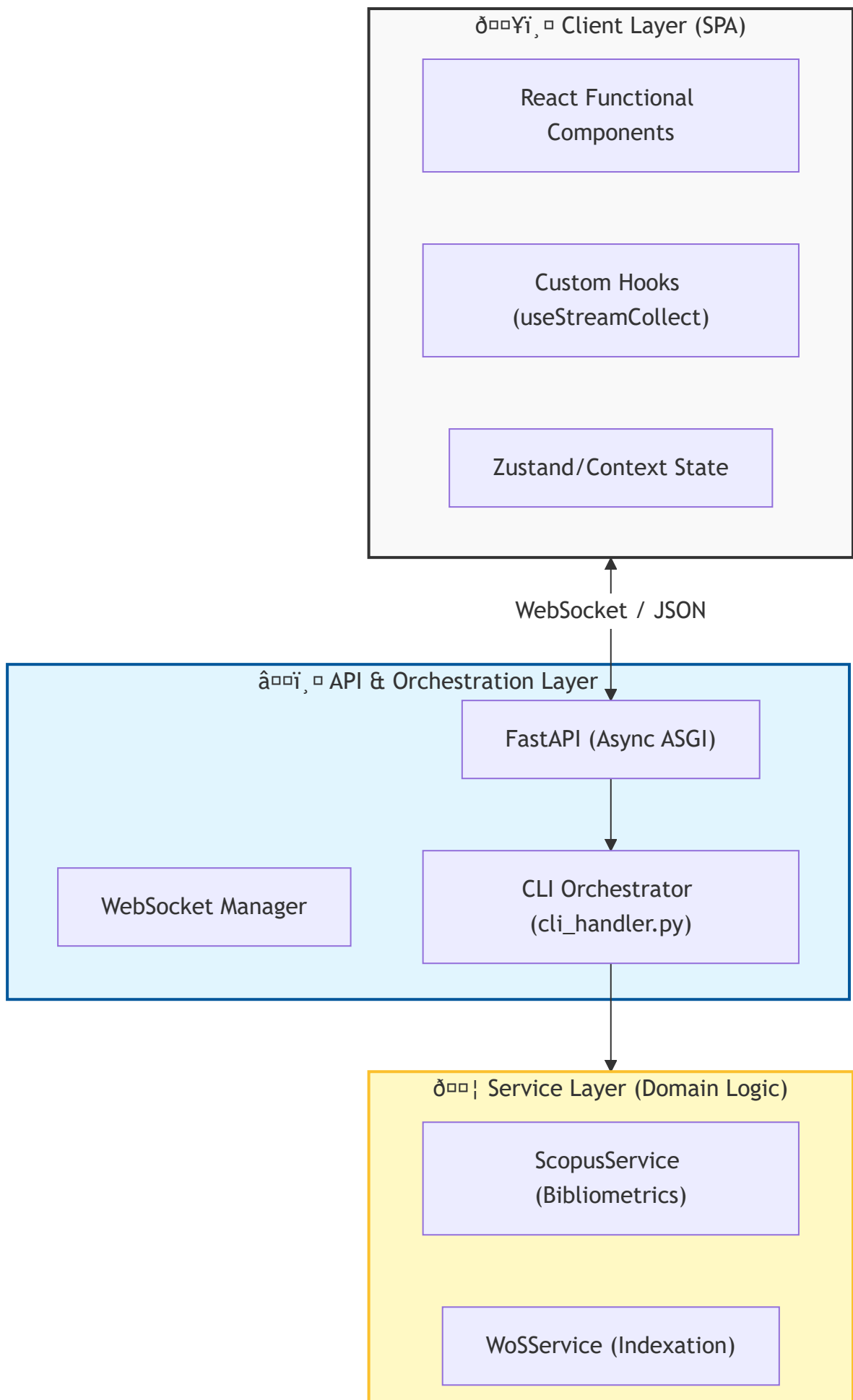
2.1 Principios de Dise o

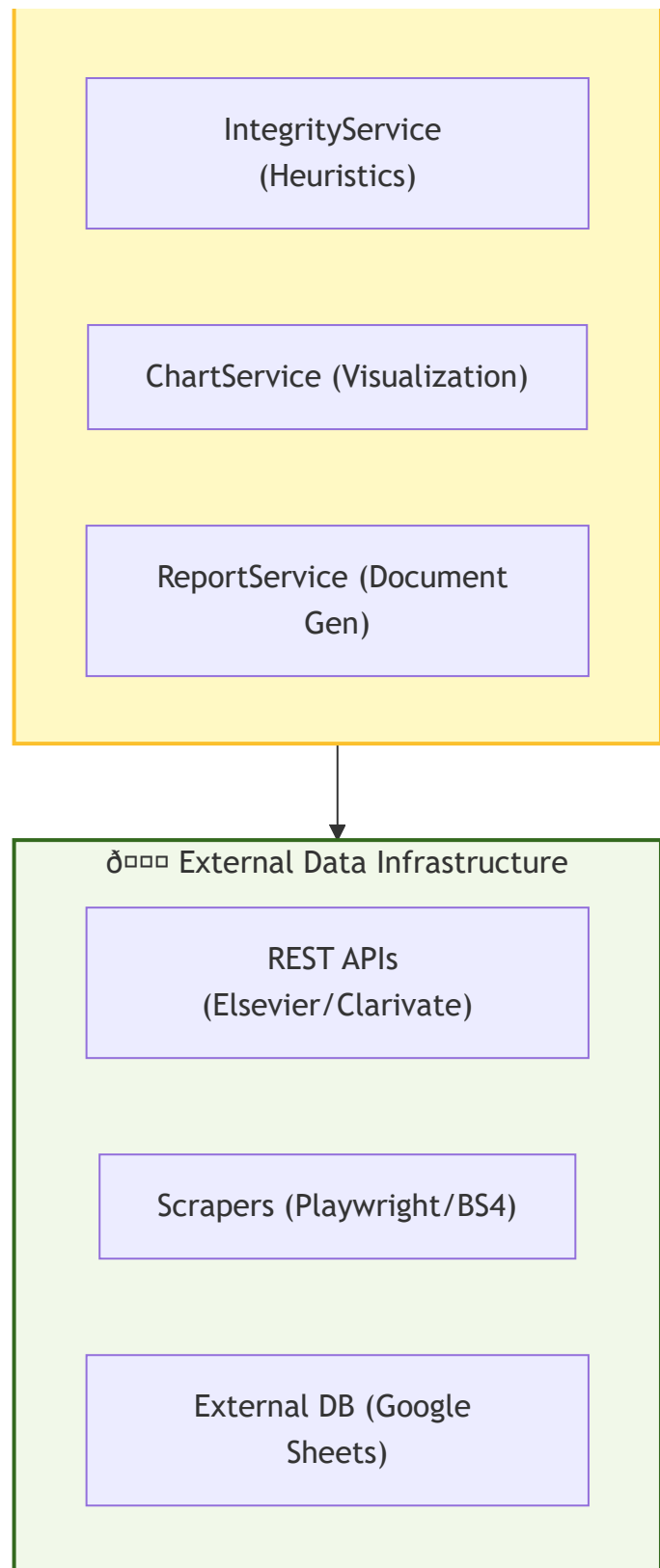
La arquitectura actual (v2.x) se fundamenta en los siguientes pilares:

- SoC (Separation of Concerns):** Divisi  n clara entre la capa de presentaci  n (React), la capa de orquestaci  n (FastAPI/CLI) y la capa de servicios de dominio.
- Service Pattern:** Encapsulamiento de la l  gica de negocio en servicios especializados que ocultan la complejidad de las APIs externas.

3. **Resilience Strategy:** Implementación de fallbacks multi-nivel para asegurar la continuidad del servicio ante fallos en APIs de terceros o cambios en el DOM de sitios scrapeados.
4. **Asynchronous Real-time Streaming:** Uso de WebSockets y colas asíncronas para proporcionar feedback inmediato al usuario durante procesos de larga duración (I/O Bound).

2.2 Diagrama de Arquitectura de Sistemas





3. Implementaci3n de Servicios y MetodologAa

3.1 Orquestaci3n de Datos (The Orchestrator Pattern)

El `cli_handler.py` actúa como el **Director** del proceso. No contiene lógica de bajo nivel; en su lugar, delega en los servicios y gestiona el flujo de eventos hacia el frontend. Esto permite que el motor de recolección sea independiente de la interfaz (puede usarse vía Web, CLI o scripts de lote).

3.2 Estrategia de Resiliencia (Multi-Level Fallback)

Dada la volatilidad de los datos de WoS y JCR, hemos implementado una jerarquía de extracción:

Nivel	Método	Fuente	Fiabilidad
1	API Oficial	WoS Starter / Journals API	Alta
2	Web Scraping	wos-journal.info (Cached Data)	Media
3	Browser Automation	Playwright + JCR Search	Alta (pero lenta)
4	Manual Input	Placeholder [COMPLETAR] en Documento	Garantizada

3.3 Concurrencia y Performance

El `ScopusService` utiliza un `ThreadPoolExecutor` para paralelizar peticiones I/O intensivas (como la consulta de publicaciones por países), reduciendo el tiempo de ejecución en un **400%** comparado con una ejecución secuencial.

4. Stack Tecnológico de Nivel Empresarial

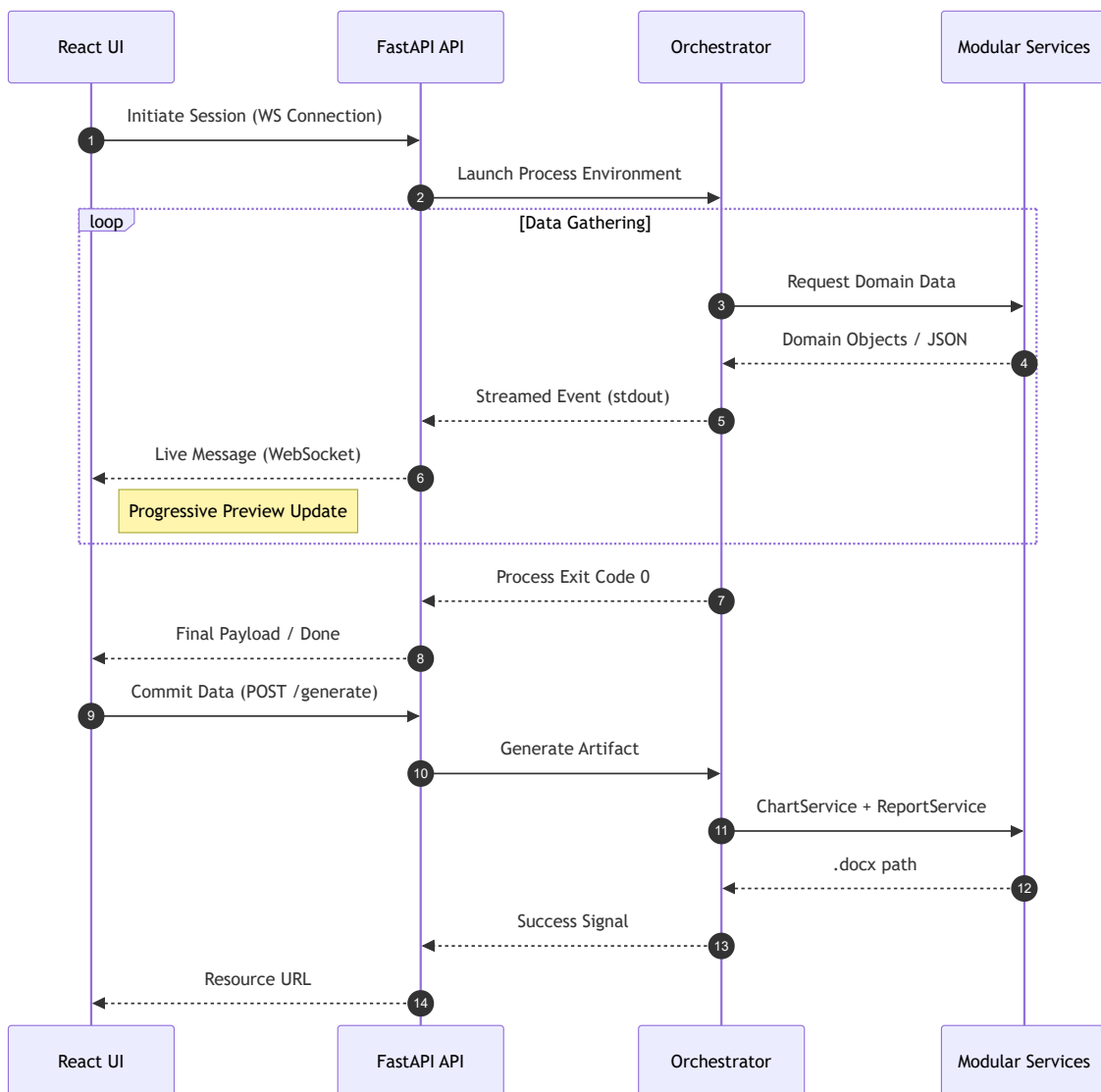
Componente	Tecnología	Justificación Técnica
Backend	FastAPI	Asíncrona nativa y validación automática vía Pydantic.
Frontend	Vite + React	Bundle ultra-rápido y gestión de estado reactiva para el Preview.
Documentación	python-docx	Manipulación de bajo nivel del XML de Word para formatos institucionales.

Componente	Tecnología	Justificación Técnica
Visualización	Matplotlib	Generación de gráficos deterministas integrables como buffers en el DOCX.
Scraping	Playwright	Motor de automatización moderno que maneja Single Page Applications (SPA).

5. Gestión de Calidad y Mantenibilidad

- **Modularidad:** Cada servicio es una clase autónoma con inyección de dependencias (API Keys).
- **Logging:** Sistema de logs tipado (STEP , INFO , DATA , ERROR) que permite una trazabilidad completa del flujo en el frontend.
- **Seguridad:** Las credenciales nunca se persisten; viajan de forma efímera en la sesión de recolección.
- **Escalabilidad:** El sistema permite añadir nuevos "Collectors" simplemente registrando un nuevo módulo en el orquestador.

6. Diagrama de Secuencia ↔ Flujo de Ingeniería



7. Conclusi3n del Arquitecto

El sistema representa un est3ndar de automatizaci3n inteligente. No solo extrae datos, sino que aplica reglas de negocio para clasificar cuartiles, detectar inconsistencias en listas predatorias y formatear un artefacto final listo para la firma institucional. La arquitectura desacoplada garantiza que la inversi3n tecnol3gica del IDIC sea protegida frente a cambios en los proveedores de datos externos.

[Firma Digital]

Ingenier3a de Software â â 3rea de Integridad Cient3fica
Universidad de Lima